

Megaprompt — YASMÍN · Identidad, acceso y RBAC

Pegar entero en Claude Code, parado en `~/Desktop/UAI/MDW-project/ecommerce-mdw`.

Sos la desarrolladora a cargo de **identidad, acceso y control de permisos** del e-commerce de BC Importados, el trabajo final de Metodologías y Desarrollo Web (UAI). Trabajás con otras tres personas: Agustín (kernel, infraestructura, inventario y pagos), Genaro (catálogo) y Nicolás (carrito, checkout y pedidos).

Hoy es domingo 04/10/2026. El Parcial I es el martes 06/10 y se defiende el backend corriendo en producción. La condición de aprobación es dura: si la URL pública no responde **o el login no funciona**, el parcial no se toma y pasa a la recuperación del 24/11. El login es tuyo: tu módulo es la mitad de esa condición.

Además, "Auth y roles" pesa **20% de la rúbrica del Parcial I**, y el nivel destacado dice *"autorización verificada en el servidor en todos los caminos, incluidos los que no se ven en la UI"*. La primera pregunta de la defensa, casi siempre, es: **"mostrame qué pasa si le pido esto con la cuenta de otro"**.

Tu bloque 2 —el middleware— es **bloqueante para las otras tres personas**: hasta que no esté pusheado, nadie puede proteger un endpoint. Hacelo temprano.

Estado real del repositorio

- Ya existe `backend/config/viper.go`, que es el **único** lugar del backend donde se leen variables de entorno (ADR-001). Si necesitás una variable nueva, se agrega como campo con su tag `mapstructure` ahí y se documenta en el ADR. Ya están declaradas `APP_SECRET_KEY` y `BOOTSTRAP_MASTER_KEY`.
- Agustín está subiendo el **kernel**: `cmd/api/main.go`, la conexión a Postgres, el migrador y el paquete `apierr`. Esperá a que esté mergeado en `develop` antes de escribir handlers; el dominio y la migración podés empezarlos ya.
- `docs/specs.md` tiene las historias `SPEC-H01`, `SPEC-H02` y `SPEC-H03`, que son las tuyas, y la sección 6 "Identidad y autorización" con las reglas.
- `docs/erd.dbml` tiene las tablas `usuario`, `rol`, `permiso`, `usuario_rol`, `rol_permiso`.

Decisiones ya tomadas que tenés que respetar

- Go 1.26 · Chi · GORM · gormigrate · PostgreSQL (ADR-001, 002 y 003).
- Identificadores** `uuid.UUID`, columnas `uuid`. Nunca autoincremental.
- Nombres de tablas y columnas en castellano y `snake_case`, como el ERD.
- Capas: el handler traduce HTTP y nada más; las reglas viven en `usecases`; las consultas en `repositories`. Un handler no arma un `db.Where(...)`.
- Contraseñas con **bcrypt**, costo 12. Nunca en texto plano, nunca en un log, nunca en una respuesta.

Reglas duras

- Sólo tocás los archivos que esta consigna te asigna.** No edites archivos de Agustín, Genaro ni Nicolás. `go.mod`, `main.go`, `router.go` y `registry.go` son de Agustín: si te falta una dependencia, pedísela por el grupo.
- `go build ./...` y `go vet ./...` en verde antes de cada push.

3. **La identidad baja de la sesión, nunca del body.** Si `usuarioID`, `rol` o `permiso` aparecen en un DTO, en un header propio o en la query, hay un agujero. Es el antipatrón que la cátedra marca explícitamente.
4. **Toda consulta que devuelve datos de un usuario lleva su id en el `WHERE`.** No un `if` comparando ids después de traer la fila: para cuando hiciste el `if`, el dato ajeno ya salió de la base y está en memoria del servidor.
5. Cada commit en español, en imperativo, y es una pieza que compila. Al terminar cada bloque: `git add`, `git commit`, `git push`. **No acumules trabajo sin pushear:** la nota es individual y se mide sobre el historial del repositorio.
6. Rama: `feat/identidad`, salida de `develop`. `main` está protegida.

```
git fetch --all
git checkout develop && git pull
git checkout -b feat/identidad
```

BLOQUE 1 — Dominio y migración

1.1 Entidades — `backend/internal/core/domain/identidad.go`

Del ERD, recortado a lo que el Parcial I necesita:

Usuario	id, nombre, apellido, correo (único), credencial_hash, estado (ACTIVO INACTIVO), origen (PUBLICO ADMINISTRATIVO), es_dueno_inicial bool, creado_en, actualizado_en	
Rol	id, nombre (único), creado_en	
Permiso	id, accion (único), descripcion	
UsuarioRol	usuario_id, rol_id	← PK compuesta (N-N)
RolPermiso	rol_id, permiso_id	← PK compuesta (N-N)

`usuario_rol` y `rol_permiso` son **las relaciones N-N que el núcleo obligatorio de la materia exige**. Que estén bien modeladas es parte del 30% del criterio "Modelo de datos", así que no las resuelvas con un campo `rol string` en `usuario`.

`credencial_hash` nunca se serializa a JSON: ponelo `json:"-"`.

1.2 Migración — `migrations/schema/identidad.go`

Agustín dejó un stub con `func MigracionesIdentidad() []*gormigrate.Migration` devolviendo vacío. Reemplazá su contenido. **Ese archivo es tuyo a partir de ahora.**

Una sola migración, ID `20261004_01_identidad`. Corre primera de las cuatro porque todas las demás tienen foreign keys hacia `usuario`.

Obligatorio:

- Índice único real en `usuario(correo)`. No alcanza con chequearlo en el código: el chequeo da una respuesta linda (409 "ese correo ya está registrado") y la restricción es la que garantiza que el dato nunca quede mal, aunque mañana alguien escriba un segundo endpoint y se olvide del chequeo. Son dos capas y las dos hacen falta.
- Índices en `usuario_rol(usuario_id)`, `usuario_rol(rol_id)`, `rol_permiso(rol_id)`.

- `onDelete` decidido explícitamente, con el motivo en un comentario:
 - `usuario_rol` y `rol_permiso` → `CASCADE` (son tablas de unión: si se borra el rol, la fila de unión no tiene sentido).
 - cualquier FK que apunte a `usuario` desde otro módulo → `RESTRICT` (un usuario con pedidos no se borra, se desactiva).

```
git add backend/internal/core/domain/identidad.go \
        backend/internal/adapters/db/postgresql/migrations/schema/identidad.go
git commit -m "feat(identidad): entidades de usuario, rol y permiso con las dos relaciones N-N"
git push -u origin feat/identidad
```

1.3 Repositorios — `repositories/identidad/`

- `usuarios.go`: `Crear`, `PorCorreo`, `PorID`, `Listar(limit, offset, filtroEstado)`, `ActualizarDatos`, `CambiarEstado`, `ExisteDueno()` `bool`.
- `roles.go`: `Crear`, `Listar`, `PorNombre`, `AsignarAUsuario`, `QuitarDeUsuario`, `RolesDeUsuario(usuarioID)`.
- `permisos.go`: `Listar`, `PermisosDeUsuario(usuarioID) []string` — una sola consulta con dos joins, **no** un `for` que consulta por cada rol. Ese `for` es el N+1 de la clase 3 y se pregunta.

Todo listado con `Limit` y `Offset` explícitos, y `Select` de las columnas que se van a usar: nunca devuelvas la fila entera de `usuario`, porque arrastra el `credencial_hash`.

```
git add backend/internal/adapters/db/postgresql/repositories/identidad/
git commit -m "feat(identidad): repositorios de usuarios, roles y permisos sin consultas N+1"
git push
```

BLOQUE 2 — El middleware (BLOQUEANTE, hazelo temprano)

Agustín dejó un stub en `backend/internal/adapters/rest/middleware/middleware.go` que devuelve 401 siempre. **Reemplazalo. A partir de acá ese archivo es tuyo.**

Esta firma está congelada: Agustín, Genaro y Nicolás ya están programando contra ella. No la cambies sin avisar por el grupo.

```

package middleware

type Usuario struct {
    ID        uuid.UUID
    Correo    string
    Roles     []string
    Permisos  []string
}

func Nuevo(secreto string) *Middleware

func (m *Middleware) RequiereSesion(next http.Handler) http.Handler
func (m *Middleware) RequierePermiso(permiso string) func(http.Handler) http.Handler
func (m *Middleware) SesionOpcional(next http.Handler) http.Handler

func UsuarioDeContexto(ctx context.Context) (Usuario, bool)

```

Cómo funciona cada uno:

- `RequiereSesion`: lee el JWT del header `Authorization: Bearer <token>` o de la cookie `sesion`. Si no hay, o está vencido, o la firma no valida → responde **401** y corta. Si está bien, mete el `Usuario` en el contexto y sigue.
- `RequierePermiso(p)`: corre después de `RequiereSesion`. Si el usuario no tiene `p` entre sus permisos → **403**. Nunca 401: 401 le dice al cliente "andá a loguearte" y 403 le dice "no insistas". Devolver uno por el otro manda a la pantalla de login a alguien que ya entró, en un loop que no termina.
- `SesionOpcional`: si hay token válido lo pone en el contexto; si no hay, deja pasar igual sin usuario. Lo necesita Nicolás para el checkout de invitado.

Los permisos viajan adentro del JWT, no se consulta la base en cada request. Es rápido, pero tiene una consecuencia que vas a tener que contestar en la defensa: si el dueño le cambia los roles a alguien, ese cambio recién aplica cuando esa persona vuelve a iniciar sesión. Eso va escrito en `docs/adr/ADR-006-sesion.md` (lo escribís vos, coordinando con Agustín): qué ganamos, qué resignamos, y por qué lo elegimos igual.

El adaptador de JWT va en `backend/internal/adapters/jwt/`:

```

func NuevoEmisor(secreto string, duracion time.Duration) *Emisor
func (e *Emisor) Emitir(u Claims) (string, error)
func (e *Emisor) Verificar(token string) (Claims, error)

```

`Claims`: `sub` (id de usuario), `correo`, `roles`, `permisos`, `exp` (24 h), `iat`. Algoritmo HS256 con `APP_SECRET_KEY`.

Verificá explícitamente el método de firma al parsear: si aceptás `alg: none` cualquiera se fabrica un token de dueño. Es el agujero clásico de JWT.

Test obligatorio `middleware_test.go`:

- Sin token → 401.
- Token con firma inválida → 401.
- Token vencido → 401.
- Token válido sin el permiso pedido → 403.
- Token válido con el permiso → pasa y el usuario está en el contexto.
- Token firmado con `alg: none` → 401.

```
git add backend/internal/adapters/jwt/ backend/internal/adapters/rest/middleware/
git commit -m "feat(identidad): middleware de sesion y permisos con JWT firmado y verificado"
git push
```

Avisá al grupo apenas esto esté mergeado en `develop` . Los tres están esperando.

BLOQUE 3 — Bootstrap del dueño inicial

SPEC-H01 . Es la operación que arranca el sistema y es una de las reglas de negocio que la IA no puede adivinar, así que es buen material de defensa.

POST `/api/auth/bootstrap` · **público a propósito**, pero sólo funciona una vez:

1. Si ya existe algún usuario con rol `dueno` → **409** `{"error":"El sistema ya fue inicializado"}` .
2. El body trae nombre, apellido, correo, contraseña y `clave_bootstrap` .
3. `clave_bootstrap` se compara contra `cfg.BootstrapMasterKey` con `subtle.ConstantTimeCompare` . Si no coincide → **403** `{"error":"Clave de inicialización inválida"}` , **sin revelar cuál era la esperada** y sin escribirla en ningún log.
4. Crea el usuario con `origen = ADMINISTRATIVO` , `es_dueno_inicial = true` y el rol `dueno` .
5. Todo adentro de **una transacción**, y con un bloqueo o una restricción que impida que dos requests simultáneos creen dos dueños iniciales (la spec lo pide explícitamente: "serializar el bootstrap"). Lo más simple: un índice único parcial sobre `es_dueno_inicial = true` .

Semilla de roles y permisos, en la misma migración o en un comando de seed propio (`backend/cmd/seed-identidad/`):

```
roles:      dueno · cliente
permisos:   usuarios.gestionar · catalogo.escribir · stock.gestionar · pedidos.gestionar
dueno      → los cuatro permisos
cliente    → ninguno (sólo compra)
```

Esos dos roles son los **"al menos 2 roles que puedan hacer cosas distintas"** del núcleo obligatorio. `dueno` administra; `cliente` compra y ve lo suyo.

```
git add backend/internal/usecases/identidad/ \
        backend/internal/adapters/rest/handlers/identidad/ \
        backend/cmd/seed-identidad/
git commit -m "feat(identidad): bootstrap protegido del dueno inicial y semilla de roles y permisos"
git push
```

BLOQUE 4 — Registro, login y cuenta

SPEC-H02 .

Método y ruta	Quién	Qué hace	Errores
POST /api/auth/registro	público	Crea una cuenta de cliente	400, 409
POST /api/auth/login	público	Devuelve el JWT	400, 401
POST /api/auth/logout	con sesión	Invalida la cookie	401
GET /api/cuenta	con sesión	Devuelve los datos propios	401
PATCH /api/cuenta	con sesión	Edita nombre, apellido, teléfono	400, 401

Las que deciden la nota:

- **El registro público crea siempre el rol de menor privilegio.** El body **no** puede traer `rol`, `roles`, `permisos` ni `origen`: si esos campos existen en el DTO, cualquiera se registra como dueño desde Postman. Esta es literalmente la regla que la spec declara crítica. El DTO sólo tiene nombre, apellido, correo y contraseña, y punto.
- **El login no enumera.** Si el correo no existe o la contraseña está mal, el mensaje es el mismo: `{"error":"Correo o contraseña incorrectos"}` con **401**. Decir "el correo existe pero la contraseña está mal" le confirma a cualquiera qué direcciones están registradas. Es peor experiencia de usuario y es a propósito: la cátedra lo marca como el contraejemplo del 409 que sí enumera.
- Comparará la contraseña con `bcrypt.CompareHashAndPassword` **siempre**, incluso cuando el correo no existe, usando un hash dummy. Si no, el tiempo de respuesta delata qué correos existen.
- Un usuario con `estado = INACTIVO` no puede iniciar sesión → 401 con el mismo mensaje genérico.
- `GET /api/cuenta` devuelve los datos **del usuario del contexto**, nunca de un id que venga en la URL o la query.

Validación con `validator/v10` sobre el DTO, y el error se traduce a `apierr.ErrValidacion` con el mapa campo → mensaje. Los mensajes van en castellano y dicen qué hacer: "La contraseña tiene que tener al menos 8 caracteres", no "invalid input".

```
git add backend/internal/usecases/identidad/ \
    backend/internal/adapters/rest/handlers/identidad/ \
    backend/internal/adapters/rest/dtos/identidad/
git commit -m "feat(identidad): registro publico como cliente, login sin enumeracion y gestion de cuenta"
git push
```

BLOQUE 5 — Administración de usuarios y roles

SPEC-H03 . Todo bajo `mw.RequierePermiso("usuarios.gestionar")` .

Método y ruta	Qué hace	Errores
GET /api/admin/usuarios	Listado paginado: nombre, correo, estado, roles. Nunca el hash	401, 403
POST /api/admin/usuarios	Crea una cuenta administrativa con <code>origen = ADMINISTRATIVO</code>	400, 401, 403, 409
PATCH /api/admin/usuarios/{id}/estado	Activa o desactiva	400, 401, 403, 404, 409
GET /api/admin/roles	Roles con sus permisos	401, 403

Método y ruta	Qué hace	Errores
POST /api/admin/roles	Crea un rol	400, 401, 403, 409
PUT /api/admin/usuarios/{id}/roles	Reemplaza los roles de una cuenta	400, 401, 403, 404

La regla de negocio que hay que implementar y que es la mejor de tu módulo para la defensa:

El dueño inicial está protegido. Ningún otro administrador puede desactivarlo, eliminarlo ni quitarle el rol `dueno`. Tampoco puede desactivarse a sí mismo.

Se valida **en el backend**, no en la interfaz, y devuelve **409** con `{"error":"El dueño inicial no puede ser modificado"}`. Escribí el test.

Esa validación vive en una **función pura** en `backend/internal/usecases/identidad/reglas.go`:

```
// Sin GORM, sin net/http, sin time.Now() adentro: todo entra por parámetro.
func PuedeCambiarEstado(actor Usuario, objetivo Usuario, nuevoEstado string) error
func PuedeAsignarRoles(actor Usuario, objetivo Usuario, roles []string) error
```

Devuelven `nil` o `apierr.ErrRegla`. Se testean llamándolas, sin levantar nada. Casos mínimos: el dueño inicial no se puede desactivar; un administrador no se puede desactivar a sí mismo; al dueño inicial no se le puede quitar el rol `dueno`; un administrador común sí puede desactivar a otro administrador común.

```
git add backend/internal/usecases/identidad/reglas.go \
        backend/internal/usecases/identidad/reglas_test.go \
        backend/internal/adapters/rest/handlers/identidad/
git commit -m "feat(identidad): administracion de cuentas y roles con proteccion del dueno inicial"
git push
```

BLOQUE 6 — Pruebas y cierre

6.1 docs/api/identidad.http

@base apuntando a `http://localhost:8080`, con la línea de producción comentada al lado. Por cada endpoint, **tres requests**: la que anda, la del body inválido (400, no 500) y la del recurso que no existe (404). Arriba de cada una, el status esperado escrito.

Y las tres que en la defensa se piden primero:

```

### sin token → 401
GET {{base}}/api/cuenta

### con el token de un CLIENTE, listando usuarios → 403
GET {{base}}/api/admin/usuarios
Authorization: Bearer {{tokenCliente}}

### con el token de un cliente, pidiendo el pedido de OTRO → 404
GET {{base}}/api/pedidos/{{idDeUnPedidoAjeno}}
Authorization: Bearer {{tokenCliente}}

```

Los tokens van en `docs/.env` (que está en el `.gitignore`) y se leen con `$dotenv`. Subí un `docs/.env.example` vacío para que los demás sepan qué completar. **Nunca commitees un token: es una sesión viva.**

6.2 Repaso del contrato, fila por fila

Abrí `docs/api/` de los cuatro módulos y verificá que **ningún endpoint quedó sin `RequiereSesion`** salvo los que son públicos a propósito, que en este sistema son exactamente:

```

POST /api/auth/registro      POST /api/auth/login      POST /api/auth/bootstrap
GET  /api/catalogo/*        GET  /api/variantes/{id}
POST /api/pagos/webhook     GET  /api/salud
GET  /api/pedidos/publico/{token}

```

Si hay otro, o le falta el middleware o le falta el comentario que diga por qué es abierto. La diferencia entre las dos cosas es una línea en el contrato, y es exactamente lo que se pregunta: "*¿este endpoint es abierto o se les olvidó?*".

6.3 ADR-006

`docs/adr/ADR-006-sesion.md`, dos párrafos: proveedor de identidad (login propio con bcrypt, no OAuth) y estrategia de sesión (JWT con permisos adentro, no sesión en base). Para cada uno: qué ganamos y qué resignamos. Agregalo a la tabla de `docs/adr/README.md`.

Con login propio, lo que se pregunta en la defensa es: cómo se hashea la contraseña, qué pasa con un correo repetido y cuánto dura la sesión. Tenelo contestado.

6.4 Contra producción

Después del merge, descomentá la `@base` de producción y corré `docs/api/identidad.http` entero. **Un login que anda en localhost y no en la URL pública, para esta materia, no existe** — y además es la condición de aprobación del parcial.

```

git add docs/api/identidad.http docs/.env.example docs/adr/
git commit -m "docs(identidad): requests de prueba, ADR-006 de sesion y revision del contrato"
git push
gh pr create --base develop --title "Identidad, acceso y RBAC" --body "Usuarios, roles y permisos con relaciones N-N, bootstrap protegido, login sin enumeración, middleware de sesión y permisos, y administración de cuentas con protección del dueño inicial."

```

Definición de terminado

No te detengas hasta que todo esto sea cierto:

- ☐ `go build ./... && go vet ./... && go test ./...` en verde.
- ☐ La migración de identidad aplicada en la base de la nube.
- ☐ Me registro, hago login y recibo un token. Con ese token entro a `/api/cuenta`.
- ☐ Con el token de un **cliente**, `GET /api/admin/usuarios` devuelve **403**.
- ☐ Sin token, cualquier endpoint protegido devuelve **401**.
- ☐ Un registro que manda `{"rol": "dueno"}` en el body crea igual un **cliente**.
- ☐ El bootstrap corrido dos veces devuelve 409 la segunda.
- ☐ El dueño inicial no se puede desactivar, ni siquiera por otro dueño: 409.
- ☐ `docs/api/identidad.http` corre entero **contra producción**.
- ☐ Todo pusheado, con commits repartidos y a tu nombre.

Si te trabás

- Si el kernel de Agustín todavía no está en `develop`, seguí con el dominio, la migración y los repositorios: no dependen de él. Los handlers sí.
- Si una migración falla, **nunca** edites una ya aplicada ni corras un reset contra la base de la nube. Se corrige con una migración nueva.
- Si algo del alcance no llega, es preferible un endpoint menos que uno que devuelve 500. Recortalo, anotalo en la spec y decilo en la defensa: eso suma, un bug escondido resta.

Lo que te van a preguntar el martes

1. **Mostrame qué pasa si le pido esto con la cuenta de otro.** (Es la primera, casi siempre.)
2. ¿Dónde se verifica el permiso, en el servidor o en el cliente? Mostrámelo sin usar la interfaz.
3. ¿Qué pasa si alguien se registra mandando `rol: "dueno"` en el body?
4. ¿Por qué el login no te dice si el correo existe?
5. ¿Cómo hashean la contraseña y por qué ese algoritmo?
6. Si el dueño le cambia el rol a alguien, ¿cuándo aplica el cambio?
7. ¿Por qué el rol está en una columna y los permisos en una tabla, y no todo en una tabla de permisos por recurso?